

PROJECT REPORT

Laravel Blog Management System

Developed By: MD. MOKLAASUR RAHAMAN

Email: rahat830611@gmail.com

Batch: WDF-1

Technology Stack: Laravel 11/12, Tailwind CSS, MySQL

Date: ____/____/____

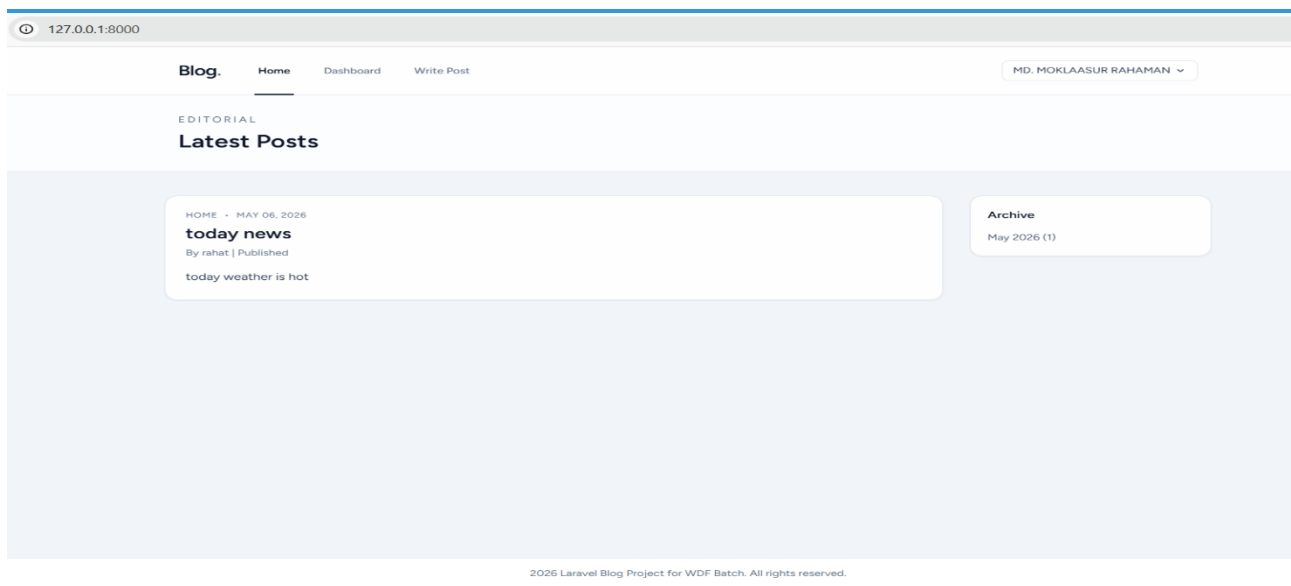
1. Introduction

This project is a modern blogging platform developed using the Laravel framework. It provides a comprehensive solution for managing blog posts, categories, tags, and user profiles. The system is designed with a focus on usability, security, and a clean user interface using Tailwind CSS.

Core Features:

- Authentication (Login, Register, Password Reset).
- Post CRUD operations with status management (Draft/Published).
- Dynamic Category and Tagging system.
- User Profile and Security settings.
- Relational Database architecture.

2. Home Page Interface

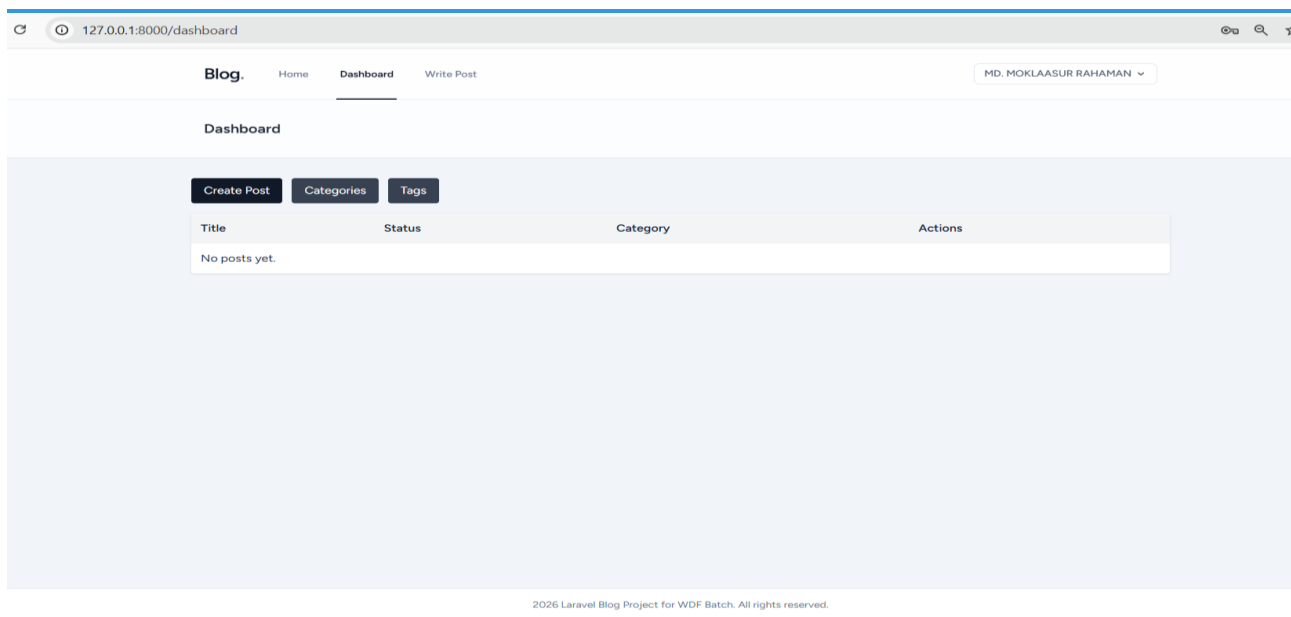


Implementation Code:

The Home Page fetches published posts and organizes them with an archive list.

```
// PostController.php
public function index() {
    $posts = Post::published()->with(['user', 'category'])->latest()->paginate(10);
    $archives = Post::published()->selectRaw('YEAR(published_at) year...')-
    >groupBy('year')->get();
    return view('posts.index', compact('posts', 'archives'));
}
```

3. User Dashboard



Logged-in users can manage their content through this private dashboard.

```
// DashboardController.php
public function index() {
    $posts = auth()->user()->posts()->latest()->paginate(10);
    return view('dashboard.index', compact('posts'));
}
```

4. Add Category

127.0.0.1:8000/categories/create

Blog. Home Dashboard Write Post MD. MOKLAASUR RAHAMAN ▾

Category Form

Name

Slug

Save

Implementation Code:

```
<?php

namespace App\Http\Controllers;

use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Str;

class CategoryController extends Controller
{
    public function index()
    {
        return view('categories.index', [
            'categories' => Category::latest()->paginate(15),
        ]);
    }

    public function create()
```

```

{
    return view('categories.create');
}

public function store(Request $request)
{
    $data = $request->validate([
        'name' => ['required', 'string', 'max:255', 'unique:categories,name'],
        'slug' => ['nullable', 'string', 'max:255', 'unique:categories,slug'],
    ]);

    $data['slug'] = $data['slug'] ?? Str::slug($data['name']);
    Category::create($data);

    return redirect()->route('categories.index')->with('success', 'Category
created successfully.');
```

```

    }

    public function edit(Category $category)
    {
        return view('categories.edit', compact('category'));
    }

    public function update(Request $request, Category $category)
    {
        $data = $request->validate([
            'name' => ['required', 'string', 'max:255', 'unique:categories,name,' .
$category->id],
            'slug' => ['nullable', 'string', 'max:255', 'unique:categories,slug,' .
$category->id],
        ]);

        $data['slug'] = $data['slug'] ?? Str::slug($data['name']);
        $category->update($data);

        return redirect()->route('categories.index')->with('success', 'Category
updated successfully.');
```

```

    }

    public function destroy(Category $category)

```

```
{  
    $category->delete();
```

```
        return redirect()->route('categories.index')->with('success', 'Category  
deleted successfully.');
```

5. Content Creation (Create Post)

The screenshot shows a web application interface for creating a new post. The browser address bar displays '127.0.0.1:8000/posts/create'. The navigation bar includes links for 'Blog.', 'Home', 'Dashboard', and 'Write Post', with the user 'MD. MOKLAASUR RAHAMAN' logged in. The main heading is 'Create Post'. The form itself contains the following elements:

- Title**: A text input field.
- Slug (optional)**: A text input field.
- Content**: A large text area for the post content.
- Featured Image URL**: A text input field.
- Select Category**: A dropdown menu.
- Tags**: A section header for the tags input.
- Draft**: A dropdown menu for the post status.
- Publish**: A button to submit the post.

A comprehensive form to handle post title, content, category, and tags.

```
// PostController.php @store
public function store(Request $request) {
    $validated = $request->validate([...]);
    $post = Post::create($validated + ['user_id' => auth()->id()]);
    $post->tags()->attach($request->tags);
    return redirect('/dashboard');
}
```

6. Profile & Security

127.0.0.1:8000/profile

Blog. Home Dashboard Write Post MD. MOKLAASUR RAHAMAN ▾

Profile Information

Update your account's profile information and email address.

Name

Email

SAVE

Update Password

Ensure your account is using a long, random password to stay secure.

Current Password

New Password

Confirm Password

SAVE

Allows users to update their personal information and secure their accounts with new passwords.

<?php

```
namespace App\Http\Controllers;

use App\Http\Requests\ProfileUpdateRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Redirect;
use Illuminate\View\View;

class ProfileController extends Controller
{
    /**
     * Display the user's profile form.
     */
    public function edit(Request $request): View
    {
        return view('profile.edit', [
            'user' => $request->user(),
        ]);
    }

    /**
     * Update the user's profile information.
     */
    public function update(ProfileUpdateRequest $request): RedirectResponse
    {
        $request->user()->fill($request->validated());

        if ($request->user()->isDirty('email')) {
            $request->user()->email_verified_at = null;
        }

        $request->user()->save();

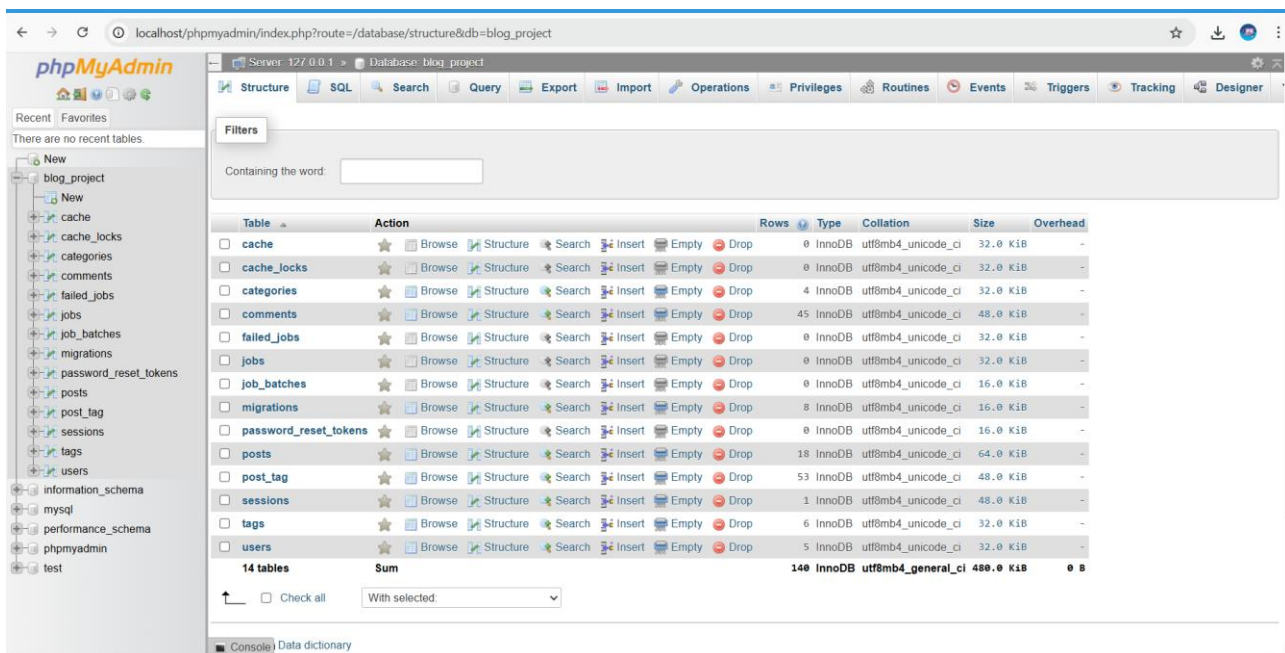
        return Redirect::route('profile.edit')->with('status', 'profile-updated');
    }

    /**
```

```
* Delete the user's account.  
*/
```

```
public function destroy(Request $request): RedirectResponse  
{  
    $request->validateWithBag('userDeletion', [  
        'password' => ['required', 'current_password'],  
    ]);  
  
    $user = $request->user();  
  
    Auth::logout();  
  
    $user->delete();  
  
    $request->session()->invalidate();  
    $request->session()->regenerateToken();  
  
    return Redirect::to('/');  
}
```

7. Database Architecture



The screenshot shows the phpMyAdmin interface for a database named 'blog_project'. The left sidebar lists the database structure, including tables like 'cache', 'cache_locks', 'categories', 'comments', 'failed_jobs', 'jobs', 'job_batches', 'migrations', 'password_reset_tokens', 'posts', 'post_tag', 'sessions', 'tags', 'users', 'information_schema', 'mysql', 'performance_schema', 'phpmyadmin', and 'test'. The main panel displays a table structure for 'blog_project' with 14 tables. Each table row includes a checkbox, the table name, an 'Action' column with icons for Browse, Structure, Search, Insert, Empty, and Drop, and columns for Rows, Type, Collation, Size, and Overhead. The tables are: cache, cache_locks, categories, comments, failed_jobs, jobs, job_batches, migrations, password_reset_tokens, posts, post_tag, sessions, tags, and users. A summary row at the bottom shows 14 tables, a total of 140 rows, and a total size of 480.0 KiB.

| Table | Action | Rows | Type | Collation | Size | Overhead |
|-----------------------|---|------|--------|--------------------|-----------|----------|
| cache | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| cache_locks | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| categories | Browse Structure Search Insert Empty Drop | 4 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| comments | Browse Structure Search Insert Empty Drop | 45 | InnoDB | utf8mb4_unicode_ci | 48.0 KiB | - |
| failed_jobs | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| jobs | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| job_batches | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 16.0 KiB | - |
| migrations | Browse Structure Search Insert Empty Drop | 8 | InnoDB | utf8mb4_unicode_ci | 16.0 KiB | - |
| password_reset_tokens | Browse Structure Search Insert Empty Drop | 0 | InnoDB | utf8mb4_unicode_ci | 16.0 KiB | - |
| posts | Browse Structure Search Insert Empty Drop | 18 | InnoDB | utf8mb4_unicode_ci | 64.0 KiB | - |
| post_tag | Browse Structure Search Insert Empty Drop | 53 | InnoDB | utf8mb4_unicode_ci | 48.0 KiB | - |
| sessions | Browse Structure Search Insert Empty Drop | 1 | InnoDB | utf8mb4_unicode_ci | 48.0 KiB | - |
| tags | Browse Structure Search Insert Empty Drop | 6 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| users | Browse Structure Search Insert Empty Drop | 5 | InnoDB | utf8mb4_unicode_ci | 32.0 KiB | - |
| 14 tables | Sum | 140 | InnoDB | utf8mb4_general_ci | 480.0 KiB | 0 B |

Database Schema (Migrations):

```
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('title');
    $table->text('content');
    $table->enum('status', ['draft', 'published']);
    $table->timestamps();
});
```

8. Installation & Setup

To run this project locally, use the following commands:

```
composer install
npm install
php artisan migrate --seed
npm run build
php artisan serve
```